

Impossibility of Distributed Consensus with One Faulty Process

MICHAEL J. FISCHER, Yale University, New Haven, Connecticut

NANCY A. LYNCH, Massachusetts Institute of Technology, Cambridge, Massachusetts

MICHAEL S. PATERSON, University of Warwick, Coventry, England

Abstract. The consensus problem involves an asynchronous system of processes, some of which may be unreliable. The problem is for the reliable processes to agree on a binary value. In this paper, it is shown that every protocol for this problem has the possibility of non-termination, even with only one faulty process. By way of contrast, solutions are known for the synchronous case, the “Byzantine Generals” problem.

Categories and Subject Descriptors: C.2.2 [Computer-Communication Networks]: Network Protocols—*protocol architecture*; C.2.4 [Computer-Communication Networks]: Distributed Systems—*distributed applications; distributed databases; network operating systems*; C.4 [Performance of Systems]: Reliability, Availability, and Serviceability; F.1.2 [Computation by Abstract Devices]: Modes of Computation—*parallelism*; H.2.4 [Database Management]: Systems—*distributed systems; transaction processing*

General Terms: Algorithms, Reliability, Theory

Additional Key Words and Phrases: Agreement problem, asynchronous system, Byzantine Generals problem, commit problem, consensus problem, distributed computing, fault tolerance, impossibility proof, reliability

Journal of the Association for Computing Machinery, Vol. 32, No. 2, April 1985, pp. 374–382.

<https://doi.org/10.1145/3149.214121>

1 Introduction

The problem of reaching agreement among remote processes is one of the most fundamental problems in distributed computing and is at the core of many algorithms for distributed data processing, distributed file management, and fault-tolerant distributed applications.

A well-known form of the problem is the “transaction commit problem”, which arises in distributed database systems [6, 13, 15–17, 21–24] (see also G. LeLann, private communication, quoted in [15]). The problem is for all the data manager processes that have participated in the processing of a particular transaction to agree on whether to install the transaction’s results in the database or to discard them. The latter action might be necessary, for example, if some data managers were, for any reason, unable to carry out the required transaction processing. Whatever decision is made, all data managers must make the same decision in order to preserve the consistency of the database.

Reaching the type of agreement needed for the “commit” problem is straightforward if the participating processes and the network are completely reliable. However, real systems are subject to a number of possible faults, such as process crashes, network partitioning, and lost, distorted, or duplicated messages. One can even consider more Byzantine types of failure [5, 7, 8, 11, 14, 18, 19] in which faulty processes might go completely haywire, perhaps even sending messages according to some malevolent plan. One therefore wants an agreement protocol that is as reliable as possible in the presence of such faults. Of course, any protocol can be overwhelmed by faults that are too frequent or too severe, so the best that one can hope for is a protocol that is tolerant to a prescribed number of “expected” faults.

In this paper, we show the surprising result that no completely asynchronous consensus protocol can tolerate even a single unannounced process death. We do not consider Byzantine failures, and we assume that the message system is reliable—it delivers all messages correctly and exactly once. Nevertheless, even with these assumptions, the stopping of a single process at an inopportune time can cause any distributed commit protocol to fail to reach agreement. Thus,

Editing of this paper was performed by guest editor S. L. Graham. The Editor-in-Chief of JACM did not participate in the processing of the paper. This work was supported in part by the Office of Naval Research under Contract N00014-82-K-0154, by the Office of Army Research under Contract DAAG29-79-C-0155, and by the National Science Foundation under Grants MCS-7924370 and MCS-8116678.

This work was originally presented at the 2nd ACM Symposium on Principles of Database Systems, March 1983.

Authors’ present addresses: M. J. Fischer, Department of Computer Science, Yale University, P.O. Box 2158, Yale Station, New Haven, CT 06520; N. A. Lynch, Laboratory for Computer Science, Massachusetts Institute of Technology, 545 Technology Square, Cambridge, MA 02139; M. S. Paterson, Department of Computer Science, University of Warwick, Coventry CV4 7AL, England

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

©1985 ACM 0004-5411/85/0400-0374 \$00.75

Journal of the Association for Computing Machinery, Vol. 32, No. 2, April 1985, pp. 374–382.

存在一个故障进程时分布式共识的不可能性

迈克尔·J·费舍尔，耶鲁大学，康涅狄格州纽黑文市

南希·A·林奇，麻省理工学院，马萨诸塞州剑桥市；迈克尔·S·帕特森，英国华威大学，考文垂市

摘要：共识问题涉及一个包含若干不可靠进程的异步系统。问题要求所有可靠进程就一个二进制值达成一致。本文证明，即使仅存在一个故障进程，该问题的任何协议均存在无法终止的可能性。作为对比，同步情形及拜占庭将军问题已有已知解法。

分类与主题描述符：C.2.2 [计算机-通信网络]：网络协议——协议架构；C.2.4 [计算机-通信网络]：分布式系统——分布式应用；分布式数据库；网络操作系统；C.4 [系统性能]：可靠性、可用性与可服务性；F.1.2 [抽象设备计算]：计算模式——并行性；H.2.4 [数据库管理]：系统——分布式系统；事务处理

通用术语：算法、可靠性、理论

补充关键词与短语：一致性问题、异步系统、拜占庭将军问题、提交问题、共识问题、分布式计算、容错性、不可能性证明、可靠性

《美国计算机学会期刊》，第32卷，第2期，1985年4月，第374—382页。
<https://doi.org/10.1145/3149.214121>

1 引言

远程进程间达成一致的问题，是分布式计算中最基础的问题之一，亦是众多分布式数据处理、分布式文件管理及容错分布式应用算法的核心。

该问题的一种广为人知形式是事务提交问题，常见于分布式数据库系统[6, 13, 15–17, 21–24]（另见G. LeLann私人通信，引自文献[15]）。该问题要求所有参与某特定事务处理的数据管理器进程就是否将该事务结果安装至数据库或予以丢弃达成一致。例如，若某些数据管理器因任何原因无法执行所需事务处理，则最后一操作可能是必要的。无论作出何种决策，所有数据管理器均须作出相同决策，以维护数据库的一致性。

若参与进程及网络完全可靠，则达成提交问题所需的这类一致性是直接可行的。然而，实际系统会面临多种可能故障，例如进程崩溃、网络分区，以及消息丢失、失真或重复。人们甚至可考虑更复杂的拜占庭式故障[5, 7, 8, 11, 14, 18, 19]，即故障进程可能完全失控，甚至按某种恶意计划发送消息。因此，人们需要一种在上述故障存在时仍尽可能可靠的共识协议。当然，任何协议均可能被过于频繁或过于严重的故障所压垮，故所能期望的最佳效果，便是设计出能容忍预定数量预期故障的协议。

本文揭示了一个令人惊讶的结果：即使仅存在单个未预先宣告的进程失效，也不存在完全异步的共识协议能够容忍该失效。我们未考虑拜占庭式故障，且假设消息系统可靠——即所有消息均被正确且恰好一次地传递。然而，即便在这些假设下，单个进程在关键时刻停止仍可能导致任意分布式提交协议无法达成一致。因此，

本文编辑工作由客座编辑S. L. 格雷厄姆完成。《美国计算机学会期刊》主编未参与本文审稿流程。本工作部分得到海军研究办公室合同N00014-82-K-0154、陆军研究办公室合同DAAG29-79-C-0155，以及美国国家科学基金会资助项目MCS-7924370和MCS-8116678的支持。

本工作最初发表于1983年3月第二届美国计算机学会数据库系统原理研讨会。

作者通讯地址：M. J. 费舍尔，耶鲁大学计算机科学系，邮政信箱2158，耶鲁站，康涅狄格州纽黑文市06520；N. A. 林奇，麻省理工学院计算机科学实验室，技术广场545号，马萨诸塞州剑桥市02139；M. S. 帕特森，英国华威大学计算机科学系，考文垂市CV4 7AL

允许免费复制本材料的全部或部分内容，前提是：所复制材料不得用于直接商业盈利目的；须在复制品上清晰标注美国计算机学会版权说明、出版物标题及其出版日期；并注明复制行为经美国计算机学会许可。其他用途或重新发表，须缴纳费用和/或取得特定许可。

©1985 美国计算机学会 0004-5411/85/0400-0374 \$00.75

《美国计算机学会期刊》，第32卷，第2期，1985年4月，第374—382页。

this important problem has no robust solution without further assumptions about the computing environment or still greater restrictions on the kind of failures to be tolerated!

Crucial to our proof is that processing is completely asynchronous; that is, we make no assumptions about the relative speeds of processes or about the delay time in delivering a message. We also assume that processes do not have access to synchronized clocks, so algorithms based on time-outs, for example, cannot be used. (In particular, the solutions in [6] are not applicable.) Finally, we do not postulate the ability to detect the death of a process, so it is impossible for one process to tell whether another has died (stopped entirely) or is just running very slowly.

Our impossibility result applies to even a very weak form of the consensus problem. Assume that every process starts with an initial value in $\{0, 1\}$. A non-faulty process decides on a value in $\{0, 1\}$ by entering an appropriate decision state. All non-faulty processes that make a decision are required to choose the same value. For the purpose of the impossibility proof, we require only that some process eventually make a decision. (Of course, any algorithm of interest would require that all non-faulty processes make a decision.) The trivial solution in which, say, 0 is always chosen is ruled out by stipulating that both 0 and 1 are possible decision values, although perhaps for different initial configurations.

Our system model is rather strong so as to make our impossibility proof as widely applicable as possible. Processes are modeled as automata (with possibly infinitely many states) that communicate by means of messages. In one atomic step, a process can attempt to receive a message, perform local computation on the basis of whether or not a message was delivered to it (and if so, which one), and send an arbitrary but finite set of messages to other processes. In particular, an “atomic broadcast” capability is assumed, so a process can send the same message in one step to all other processes with the knowledge that if any non-faulty process receives the message, then all the non-faulty processes will. Every message is eventually delivered as long as the destination process makes infinitely many attempts to receive, but messages can be delayed, arbitrarily long, and delivered out of order.

The asynchronous commit protocols in current use all seem to have a “window of vulnerability”—an interval of time during the execution of the algorithm in which the delay or inaccessibility of a single process can cause the entire algorithm to wait indefinitely. It follows from our impossibility result that every commit protocol has such a “window”, confirming a widely believed tenet in the folklore.

2 Consensus Protocols

A *consensus protocol* P is an asynchronous system of N processes ($N \geq 2$). Each process p has a one-bit *input register* x_p , an *output register* y_p with values in $\{b, 0, 1\}$, and an unbounded amount of internal storage. The values in the input and output registers, together with the program counter and internal storage, comprise the *internal state*. *Initial states* prescribe fixed starting values for all but the input register; in particular, the output register starts with value b . The states in which the output register has value 0 or 1 are distinguished as being *decision states*. p acts deterministically according to a *transition function*. The transition function cannot change the value of the output register once the process has reached a decision state; that is, the output register is “write-once”. The entire system P is specified by the transition functions associated with each of the processes and the initial values of the input registers.

Processes communicate by sending each other messages. A *message* is a pair (p, m) , where p is the name of the destination process and m is a “message value” from a fixed universe M . The *message system* maintains a multiset, called the *message buffer*, of messages that have been sent but not yet delivered. It supports two abstract operations:

- $\text{send}(p, m)$: Places (p, m) in the message buffer;
- $\text{receive}(p)$: Deletes some message (p, m) from the buffer and returns m , in which case we say (p, m) is *delivered*, or returns the special null marker $\emptyset$ and leaves the buffer unchanged.

Thus, the message system acts non-deterministically, subject only to the condition that if $\text{receive}(p)$ is performed infinitely many times, then every message (p, m) in the message buffer is eventually delivered. In particular, the message system is allowed to return $\emptyset$ a finite number of times in response to $\text{receive}(p)$, even though a message (p, m) is present in the buffer.

A *configuration* of the system consists of the internal state of each process, together with the contents of the message buffer. An *initial configuration* is one in which each process starts at an initial state and the message buffer is empty.

A *step* takes one configuration to another and consists of a primitive step by a single process p . Let C be a configuration. The step occurs in two phases. First, $\text{receive}(p)$ is performed on the message buffer in C to obtain a value $m \in M \cup \{\emptyset\}$. Then, depending on p ’s internal state in C and on m , p enters a new internal state and sends a finite set of messages to other processes. Since processes are deterministic, the step is completely determined by the pair $e = (p, m)$, which we call an *event*. (This “event” should be thought of as the receipt of m by p .) $e(C)$ denotes the resulting configuration, and we say that e can be *applied* to C . Note that the event $(p, \emptyset)$ can always be applied to C , so it is always possible for a process to take another step.

这一重要问题若无关于计算环境的进一步假设, 或对容错类型施加更严格的限制, 则根本不存在鲁棒解!

我们证明的关键在于处理过程完全异步; 即我们不对进程间的相对速度或消息传递延迟时间作任何假设。我们还假设进程无法访问同步时钟, 因此基于超时机制的算法(例如)不可用。(特别地, 文献[6]中的解法不适用。)最后, 我们未预设进程死亡检测能力, 故一个进程无法判断另一进程是已终止(完全停止)还是仅运行得极慢。

我们的不可能性结果适用于共识问题的一种非常弱的形式。假设每个进程初始值均取自 $\{0, 1\}$ 。一个非故障进程通过进入相应的决策状态来决定取值 $\{0, 1\}$ 。所有作出决策的非故障进程必须选择相同值。在不可能性证明中, 我们仅要求至少存在某个进程最终作出决策。(当然, 任何有实际意义的算法均要求所有非故障进程均作出决策。)通过规定0和1均为可能的决策值(尽管可能对应不同初始配置), 排除了始终选择0等平凡解法。

我们的系统模型设定得相当强, 以使不可能性证明尽可能广泛适用。进程被建模为自动机(可能具有无穷多个状态), 通过消息进行通信。在一个原子步骤中, 进程可尝试接收一条消息, 并基于是否收到消息(若收到则基于具体消息内容)执行本地计算, 同时向其他进程发送任意但有限数量的消息。特别地, 我们假定具备原子广播能力, 因此进程可在单步内向所有其他进程发送同一消息, 并确信只要任一非故障进程接收到该消息, 则所有非故障进程均会接收到。只要目标进程无限次尝试接收, 每条消息最终必被传递; 但消息可能被任意长时间延迟, 且可能乱序传递。

当前实际使用的异步提交协议似乎均存在一个脆弱窗口——即算法执行过程中的一段时间区间, 在此期间, 单个进程的延迟或不可访问性可能导致整个算法无限期等待。我们的不可能性结果表明, 所有提交协议均必然存在此类脆弱窗口, 从而证实了业界普遍信奉的一个经验法则。

2 共识协议

共识协议是一种由(≥ 2)个进程构成的异步系统。每个进程拥有一个一位输入寄存器、一个取值于 $\{0, 1\}$ 的输出寄存器, 以及无界容量的内部存储空间。输入寄存器与输出寄存器中的值、程序计数器及内部存储空间共同构成内部状态。初始状态对除输入寄存器外的所有变量均规定了固定的起始值; 特别是, 输出寄存器起始值为。输出寄存器取值为0或1的状态被特别定义为决策状态。进程依据转移函数确定性地运行。一旦进程进入决策状态, 转移函数便不可再更改输出寄存器的值; 即输出寄存器为一次性写入型。整个系统由各进程关联的转移函数及输入寄存器的初始值共同确定。

进程通过相互发送消息进行通信。消息为一对 (i, v) , 其中 i 为目标进程名称, 为来自固定全集的消息值。消息系统维护一个称为消息缓冲区的多重集合, 用于存放已发送但尚未传递的消息。该系统支持两种抽象操作:

- $\text{send}(i, v)$: 将 (i, v) 放入消息缓冲区;
- $\text{receive}()$: 从缓冲区中删除某个消息 (i, v) 并返回, 此时称该消息 (i, v) 已被传递; 或返回特殊空标记 $\emptyset$, 且缓冲区保持不变。

因此, 消息系统以非确定性方式运行, 唯一约束条件是: 若无限次执行 $\text{receive}()$ 操作, 则消息缓冲区中每个消息 (i, v) 最终都会被传递。特别地, 即使消息缓冲区中已存在消息 (i, v) , 消息系统仍可针对 $\text{receive}()$ 操作有限次地返回特殊空标记 $\emptyset$, 且不改变缓冲区内容。

系统的某一配置由每个进程的**内部状态**以及消息缓冲区的内容共同构成。初始配置是指每个进程均从其初始状态开始, 且消息缓冲区为空的配置。

一个步骤将一个配置转变为另一个配置, 且由单个进程执行一个基本操作构成。设 C 为一个配置。该步骤分为两个阶段: 首先, 在 C 中对消息缓冲区执行 $\text{receive}()$ 操作, 得到一个值 $v \in \mathcal{V}$; 然后, 依据在 C 中的内部状态及, 该进程进入一个新的内部状态, 并向其他进程发送一个有限的消息集合。由于进程是确定性的, 该步骤完全由二元组 (C, i) 决定, 我们称其为一个事件。(该事件应理解为进程接收了。) C' 表示执行该事件后所得的配置, 我们称该事件可应用于 C 。注意, 事件 (C, i) 总能应用于 C , 因此进程总可以再执行一个步骤。

A *schedule* from C is a finite or infinite sequence σ of events that can be applied, in turn, starting from C . The associated sequence of steps is called a *run*. If σ is finite, we let $\sigma(C)$ denote the resulting configuration, which is said to be *reachable* from C . A configuration reachable from some initial configuration is said to be *accessible*. Hereafter, all configurations mentioned are assumed to be accessible.

The following lemma expresses a “commutativity” property of schedules.

LEMMA 1. *Suppose that from some configuration C , the schedules σ_1, σ_2 , lead to configurations C_1, C_2 , respectively. If the sets of processes taking steps in σ_1 and σ_2 , respectively, are disjoint, then σ_2 can be applied to C_1 and σ_1 can be applied to C_2 , and both lead to the same configuration C_3 . (See Figure 1.)*

PROOF. The result follows at once from the system definition, since σ_1 and σ_2 do not interact. $\square$

A configuration C has *decision value* v if some process p is in a decision state with $y_p = v$. A consensus protocol is *partially correct* if it satisfies two conditions:

- (1) No accessible configuration has more than one decision value.
- (2) For each $v \in \{0, 1\}$, some accessible configuration has decision value v .

A process p is *non-faulty* in a run provided that it takes infinitely many steps, and it is *faulty* otherwise. A run is *admissible* provided that at most one process is faulty and that all messages sent to non-faulty processes are eventually received.

A run is a *deciding* run provided that some process reaches a decision state in that run. A consensus protocol P is *totally correct in spite of one fault* if it is partially correct, and every admissible run is a deciding run. Our main theorem shows that every partially correct protocol for the consensus problem has some admissible run that is not a deciding run.

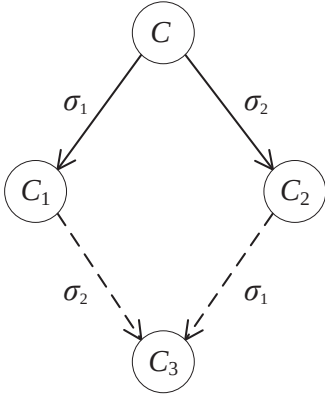


Figure 1

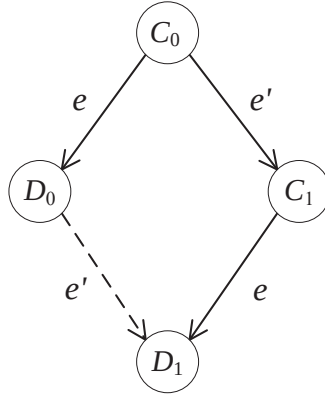


Figure 2

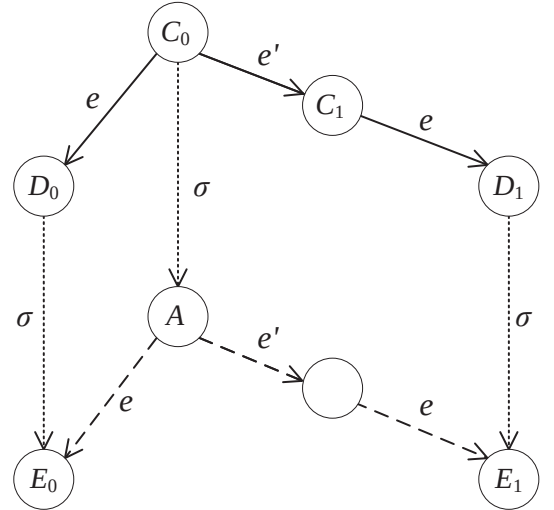


Figure 3

3 Main Result

THEOREM 1. *No consensus protocol is totally correct in spite of one fault.*

PROOF. Assume to the contrary that P is a consensus protocol that is totally correct in spite of one fault. We prove a sequence of lemmas which eventually lead to a contradiction.

The basic idea is to show circumstances under which the protocol remains forever indecisive. This involves two steps. First, we argue that there is some initial configuration in which the decision is not already predetermined. Second, we construct an admissible run that avoids ever taking a step that would commit the system to a particular decision.

Let C be a configuration and let V be the set of decision values of configurations reachable from C . C is *bivalent* if $|V| = 2$. C is *univalent* if $|V| = 1$, let us say *0-valent* or *1-valent* according to the corresponding decision value. By the total correctness of P , and the fact that there are always admissible runs, $V \neq \emptyset$. $\square$

LEMMA 2. *P has a bivalent initial configuration.*

PROOF. Assume not. Then P must have both 0-valent and 1-valent initial configurations by the assumed partial correctness. Let us call two initial configurations *adjacent* if they differ only in the initial value x_p of a single process p . Any two initial configurations are joined by a chain of initial configurations, each adjacent to the next. Hence, there must exist a 0-valent initial configuration C_0 adjacent to a 1-valent initial configuration C_1 . Let p be the process in whose initial value they differ.

从出发的一个调度，是指一个有限或无限的事件序列，该序列可依次从开始应用。相应的步骤序列称为一次执行。若为有限序列，则用 $()$ 表示其产生的配置，该配置被称为从可达的。从某个初始配置可达的配置称为可达配置。此后文中提及的所有配置均默认为可达配置。

以下引理表达了调度所具有的交流性质。

引理1：假设从某个配置出发，调度1和2分别导致配置1和2。若在1和2中执行步骤的进程集合互不相交，则2可应用于1，1亦可应用于2，且两者均导致同一配置3。（参见图1）

证明：该结论可直接由系统定义得出，因为1和2之间并无交互。□

若某个进程处于一个决策状态且其决策值为 v ，则称该配置具有决策值 v 。若一个共识协议满足以下两个条件，则称其为部分正确的：

- (1) 任意可达配置至多具有一个决策值。
- (2) 对每个 $v \in \{0, 1\}$ ，均存在某个可达配置具有决策值 v 。

若某进程在一次执行中执行了无穷多个步骤，则称其为非故障进程；否则称其为故障进程。若一次执行中至多有一个故障进程，且所有发往非故障进程的消息最终均被接收，则称该执行为容许执行。

若某次执行过程中有某个进程进入了决策状态，则称该执行为决策执行。若一个共识协议既满足部分正确性，又使得所有容许执行均为决策执行，则称其在发生一次故障时仍具有完全正确性。我们的主定理表明：对于共识问题而言，任何满足部分正确性的协议，都必然存在某个容许执行过程不是决策执行。

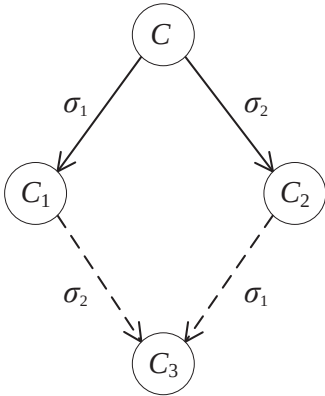


Figure 1

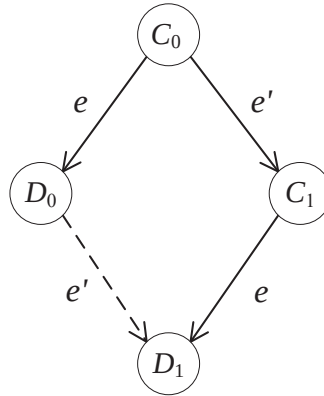


Figure 2

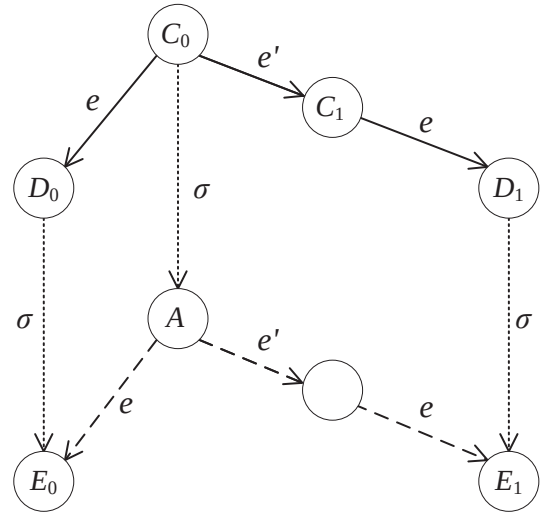


Figure 3

3 主要结果

定理1：不存在任何一个共识协议，能够在发生一次故障的情况下仍保持完全正确性。

证明：反设存在一个共识协议，它尽管发生一次故障仍具备完全正确性。我们将依次证明若干引理，最终导出矛盾。

基本思路是揭示协议可能永远无法达成决策的若干情形。这包含两个步骤：首先，论证存在某个初始配置，使得决策结果并非预先确定；其次，构造一个容许执行过程，使其在任意时刻均避免执行会导致系统承诺某一特定决策的步骤。

设 Δ 为一个配置，为所有从可达的配置所具有的决策值构成的集合。若 $|\Delta| = 2$ ，则称是双值的；若 $|\Delta| = 1$ ，则称是单值的；进一步地，若该唯一决策值为0或1，则分别称为0-单值或1-单值。根据的完全正确性以及始终存在容许执行过程这一事实， $\Delta \neq \emptyset$ 。□

引理2：存在一个双值的初始配置。

证明：反设不成立。则根据所假设的部分正确性，必同时存在取值为0的初始配置和取值为1的初始配置。若两个初始配置仅在一个进程的初始值上不同，则称它们相邻。任意两个初始配置均可由一串初始配置连接起来，其中每一对相邻配置彼此仅在一个进程的初始值上不同。因此，必然存在一个取值为0的初始配置 C_0 ，它与某个取值为1的初始配置 C_1 相邻。设为这两个配置中初始值不同的那个进程。

Now consider some admissible deciding run from C_0 in which process p takes no steps, and let σ be the associated schedule. Then σ can be applied to C_1 also, and corresponding configurations in the two runs are identical except for the internal state of process p . It is easily shown that both runs eventually reach the same decision value. If the value is 1, then C_0 is bivalent; otherwise, C_1 is bivalent. Either case contradicts the assumed nonexistence of a bivalent initial configuration. $\square$

LEMMA 3. *Let C be a bivalent configuration of P , and let $e = (p, m)$ be an event that is applicable to C . Let $\mathcal{C}$ be the set of configurations reachable from C without applying e , and let $\mathcal{D} = e(\mathcal{C}) = \{e(E) | E \in \mathcal{C} \text{ and } e \text{ is applicable to } E\}$. Then, $\mathcal{D}$ contains a bivalent configuration.*

PROOF. Since e is applicable to C , then by definition of $\mathcal{C}$ and the fact that messages can be delayed arbitrarily, e is applicable to every $E \in \mathcal{C}$.

Now assume that $\mathcal{D}$ contains no bivalent configurations, so every configuration $D \in \mathcal{D}$ is univalent. We proceed to derive a contradiction.

Let E_i be an i -valent configuration reachable from C , $i = 0, 1$. (E_i exists since C is bivalent.) If $E_i \in \mathcal{C}$, let $F_i = e(E_i) \in \mathcal{D}$. Otherwise, e was applied in reaching E_i , and so there exists $F_i \in \mathcal{D}$ from which E_i is reachable. In either case, F_i is i -valent since F_i is not bivalent (since $F_i \in \mathcal{D}$ and $\mathcal{D}$ contains no bivalent configurations) and one of E_i and F_i is reachable from the other. Since $F_i \in \mathcal{D}$, $i = 0, 1$, $\mathcal{D}$ contains both 0-valent and 1-valent configurations.

Call two configurations *neighbors* if one results from the other in a single step. By an easy induction, there exist neighbors $C_0, C_1 \in \mathcal{C}$ such that $D_i = e(C_i)$ is i -valent, $i = 0, 1$. Without loss of generality, $C_1 = e'(C_0)$ where $e' = (p', m')$.

Case 1. If $p' \neq p$, then $D_1 = e'(D_0)$ by Lemma 1. This is impossible, since any successor of a 0-valent configuration is 0-valent. (See Figure 2.)

Case 2. If $p' = p$, then consider any finite deciding run from C_0 in which p takes no steps.

Let σ be the corresponding schedule, and let $A = \sigma(C_0)$. By Lemma 1, σ is applicable to D_i , and it leads to an i -valent configuration $E_i = \sigma(D_i)$, $i = 0, 1$. Also by Lemma 1, $e(A) = E_0$ and $e(e'(A)) = E_1$. (See Figure 3.) Hence, A is bivalent. But this is impossible since the run to A is deciding (by assumption), so A must be univalent.

In each case, we reached a contradiction, so $\mathcal{D}$ contains a bivalent configuration. $\square$

Any deciding run from a bivalent initial configuration goes to a univalent configuration, so there must be some single step that goes from a bivalent to a univalent configuration. Such a step determines the eventual decision value. We now show that it is always possible to run the system in a way that avoids such steps, leading to an admissible non-deciding run.

The run is constructed in stages, starting from an initial configuration. We ensure that the run is admissible in the following way. A queue of processes is maintained, initially in an arbitrary order, and the message buffer in a configuration is ordered according to the time the messages were sent, earliest first. Each stage consists of one or more process steps. The stage ends with the first process in the process queue taking a step in which, if its message queue was not empty at the start of the stage, its earliest message is received. This process is then moved to the back of the process queue. In any infinite sequence of such stages every process takes infinitely many steps and receives every message sent to it. The run is therefore admissible. Our problem, of course, is to do this in such a way as to avoid a decision ever being reached.

Let C_0 be a bivalent initial configuration whose existence is assured by Lemma 2. Execution begins in C_0 , and we ensure that every stage begins from a bivalent configuration. Suppose then that configuration C is bivalent and that process p heads the priority queue. Let m be the earliest message to p in C 's message buffer, if any, and $\emptyset$ otherwise. Let $e = (p, m)$. By Lemma 3, there is a bivalent configuration C' reachable from C by a schedule in which e is the last event applied. The corresponding sequence of steps defines the stage.

Since each stage ends in a bivalent configuration, every stage in the construction of the infinite schedule succeeds. The resulting run is admissible, and no decision is ever reached. It follows that P is not totally correct. $\square$

4 Initially Dead Processes

In this section, we exhibit a protocol that solves the consensus problem for N processes as long as a majority of the processes are non-faulty and no process dies during the execution of the protocol. No process knows in advance, however, which of the processes are initially dead and which are not.

The protocol works in two stages. During the first stage, the processes construct a directed graph G with a node corresponding to each process. Every process broadcasts a message containing its process number and then listens for messages from $L - 1$ other processes, where $L = \lceil (N + 1)/2 \rceil$. G has an edge from i to j iff j receives a message from i . Thus, G has in-degree $L - 1$.

现在考虑从初始配置0出发的某个容许决策执行过程，其中某个进程不执行任何步骤，设其对应的调度为。那么该调度也可应用于配置1，且这两个执行过程中对应的各个状态完全相同，仅该进程的内部状态除外。容易证明，这两个执行过程最终都会到达相同的决策值。若该值为1，则配置0是双值的；否则，配置1是双值的。无论哪种情况，均与假设中不存在双值初始配置相矛盾。□

引理3：设为的一个双值构型， $\pi()$ 为一个可应用于的事件。令为所有不应用而从可达的构型之集合，令 $\pi() = \{() | \in\}$ 。则必含一个双值构型。

证明：由于可应用于，故根据及消息可被任意延迟这一事实的定义，可应用于每个 $\in$ 。

现假设不含双值构型，因此中每个构型均为单值构型。我们将由此推导出矛盾。

设为从可达的一个-值构型， $\pi = 0, 1$ 。（因是双值构型，故存在。）若 $\in$ ，则令 $\pi() \in$ 。否则，是在抵达过程中应用了，因而存在某个 $\in$ ，使得从可达。无论哪种情形，均为-值构型，因为非双值构型（因 $\in$ 且不含双值构型），且或之一可从另一个抵达。由于 $\in$ ， $\pi = 0, 1$ ，故既包含0-值构型，也包含1-值构型。

若两个构型经单步操作相互转化，则称其为邻接构型。通过简单的归纳法，可知存在邻接构型 $0, 1 \in$ ，使得 $\pi()$ 为-值构型， $\pi = 0, 1$ 。不失一般性，设 $1 = \pi'(0)$ ，其中 $\pi' = (\pi')$ 。

情形1：若 $\pi' \neq$ ，则由引理1可得 $1 = \pi'(0)$ 。这不可能，因为0-值构型的任一后继构型均为0-值构型。（参见图2。）

情形2：若 $\pi' =$ ，则考虑从0出发的任意有限决策执行过程，其中不执行任何步骤。

令为对应的调度，令 $\pi = (0)$ 。根据引理1，可应用于，且将导向一个-值构型 $\pi()$ ， $\pi = 0, 1$ 。同样依据引理1， $\pi() = 0$ 且 $\pi'(0) = 1$ 。（参见图3。）因此，为双值构型。但这不可能，因为至的执行过程是决策性的（根据假设），故必为单值构型。

在每种情形下，我们都得出了矛盾，因此必含一个双值构型。□

任何从双值初始构型出发的决策执行过程均将抵达单值构型，因此必然存在某单个步骤，其输入为双值构型而输出为单值构型。该步骤决定了最终的决策值。接下来我们将证明，始终可以以某种方式运行系统来规避此类步骤，从而导出一个可接受但永不决策的执行过程。

该执行过程按阶段构建，起始于一个初始构型。我们以如下方式确保该执行过程是可接受的：维护一个进程队列，初始顺序任意；而构型中的消息缓冲区则按消息发送时间排序，最早发送者排在最前。每一阶段包含一个或多个进程步骤。该阶段以进程队列队首的进程执行一步而告终：若其消息队列在该阶段开始时非空，则接收其中最早的一条消息；此后，该进程被移至进程队列队尾。在如此无限延续的阶段序列中，每个进程均将执行无穷多次步骤，并接收所有发给它的消息。因此，该执行过程是可接受的。当然，我们的难点在于以某种方式运行系统，从而避免任何决策的达成。

设0为引理2所保证存在的双值初始构型。执行始于0，且我们确保每一阶段均始于一个双值构型。假定构型为双值构型，且进程位于优先队列队首。令为进程中消息缓冲区里最早的一条消息（若存在），否则为空集。令 $\pi = ()$ 。根据引理3，存在一个双值构型 π' ，其可通过一个以作为最后应用事件的调度从到达。相应的一系列步骤即定义了该阶段。

由于每一阶段均以双值构型结束，因此无限调度构建过程中的每一阶段均成功完成。所得执行过程是可接受的，且从未达成任何决策。由此可知，该协议并非完全正确的。□

4 初始即失效的进程

在本节中，我们给出一种协议，只要多数进程是非故障的，且协议执行期间无进程失效，该协议即可解决个进程的共识问题。然而，任一进程均无法预先知晓哪些进程初始即为失效，哪些不是。

该协议分两个阶段运行。在第一阶段，各进程构造一个有向图，其中每个进程对应一个节点。每个进程广播一条包含其进程编号的消息，然后监听来自其余 $n-1$ 个进程的消息，其中 $n = \lceil (n+1)/2 \rceil$ 。若进程收到进程发来的消息，则图中存在一条从到的边。因此，图的入度为 $n-1$ 。

In the second stage, the processes construct G^+ (the transitive closure of G) in the sense that upon completion of this stage, each process k knows about all of the edges (j, k) incident on k in G^+ as well as the initial values of all such j .

To carry out this stage, each process broadcasts to all other processes its process number and initial value together with the names of the $L - 1$ processes it heard from during the first stage. It then waits until it has received a stage 2 message from every ancestor in G that it knows about. Initially, it knows only about the $L - 1$ processes from which it heard directly during the first stage, but it learns about additional ancestors from the stage 2 messages that it receives. Waiting continues until such time as all currently known-about processes have been heard from.

At this point, each process knows all of its own ancestors and the edges of G incident on them. Using this information, it computes all of the edges of G^+ incident on each of its ancestors. It then determines which of its ancestors belong to an *initial* clique of G^+ , that is, a clique with no incoming edges. To do this, it uses the fact that a node k is in an initial clique iff k is itself an ancestor of every node j that is an ancestor of k . Since every node in G^+ has at least $L - 1$ predecessors, there can be only one initial clique; it has cardinality at least L , and every process that completes the second stage knows exactly the set of processes comprising it.

Finally, each process makes a decision based on the initial values of the processes in the initial clique using any agreed-upon rule. Since all processes know the initial values of all members of the initial clique, they all reach the same decision.

The correctness of this protocol proves the following theorem.

THEOREM 2. *There is a partially correct consensus protocol in which all non-faulty processes always reach a decision, provided no processes die during its execution and a strict majority of the processes are alive initially.*

5 Conclusion

We have shown that a natural and important problem of fault-tolerant cooperative computing cannot be solved in a totally asynchronous model of computation. These results do not show that such problems cannot be “solved” in practice; rather, they point up the need for more refined models of distributed computing that better reflect realistic assumptions about processor and communication timings, and for less stringent requirements on the solution to such problems. (For example, termination might be required only with probability 1.) Subsequent to the original announcement of these results [12], progress has been made along both of these lines [1–4, 9, 10, 20, 25].

Acknowledgments

The authors would like to thank John Guttag for helpful discussions during the initial phase of this work, and Gene Stark for discussion of the results and a careful reading of the text. They also thank the referees for pointing out several places where the presentation needed improvement.

在第二阶段，各进程以如下方式构建图+（即图的传递闭包）：当该阶段结束时，每个进程均知晓图+中所有指向自身的边($\rightarrow$)，以及所有对应节点的初始值。

为执行该阶段，每个进程向其余所有进程广播其进程编号、初始值，以及其在第一阶段所听到的-1个进程的名称。随后，它持续等待，直至收到来自其所知悉的所有祖先节点的第二阶段消息。初始时，它仅知晓第一阶段中直接收到消息的那-1个进程；但通过接收第二阶段消息，它可获知更多祖先节点。等待将持续至当前所知悉的所有进程均已被接收到消息为止。

此时，每个进程均已知晓自身所有祖先节点及其入边。利用该信息，它进一步计算出图+中所有指向其各祖先节点的边。随后，它判定哪些祖先节点属于图+的初始团（即无任何入边的团）。为此，它利用如下事实：一个节点属于初始团，当且仅当该节点是每一个其祖先节点的祖先。由于图+中每个节点至少有-1个前驱节点，故初始团至多只有一个；其基数至少为1，且所有完成第二阶段的进程均精确知晓构成该初始团的进程集合。

最后，每个进程根据初始团中各进程的初始值，依事先约定的任意规则作出决策。由于所有进程均知晓初始团中所有成员的初始值，因此它们将达成相同的决策。

该协议的正确性证明了如下定理。

定理2：存在一种部分正确的共识协议，只要在其执行过程中无进程失效，且初始时多数进程处于活跃状态，则所有非故障进程总能达成决策。

5 结论

我们已证明，在完全异步计算模型中，容错协同计算这一自然且重要的问题无法求解。这些结果并未表明此类问题在实践中不可求解；相反，它们凸显了对分布式计算模型进行更精细刻画的必要性——该模型应能更好地反映处理器和通信时序方面的现实假设，同时也凸显了对此类问题的求解要求可适当放宽的必要性。（例如，终止性只需以概率1成立即可。）在这些结果最初公布[12]之后，上述两个方向均已取得进展[1–4, 9, 10, 20, 25]。

致谢

作者谨向约翰·古特格在本工作初期所给予的有益讨论表示感谢，并感谢吉恩·斯塔克就研究结果展开的讨论以及对本文的细致审阅。作者亦感谢各位审稿人指出了若干处需要改进表述的地方。

References

- [1] ATTIYA, C., DOLEV, D., AND GIL, J. Asynchronous Byzantine consensus. In *Proceedings of the 3rd Annual ACM Symposium on Principles of Distributed Computing* (Vancouver, B.C., Canada, Aug. 27-29). ACM, New York, 1984, pp. 119-133.
- [2] BEN-OR, M. Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols. In *Proceedings of the 2nd Annual ACM Symposium on Principles of Distributed Computing* (Montreal, Quebec, Canada, Aug. 17-19). ACM, New York, 1983, pp. 27-30.
- [3] BRACHA, G. An asynchronous $[(n-1)/3]$ -resilient consensus protocol. In *Proceedings of the 3rd Annual ACM Symposium on Principles of Distributed Computing* (Vancouver, B.C., Canada, Aug. 27-29). ACM, New York, 1984, pp. 154-162.
- [4] BRACHA, G., AND TOUEG, S. Resilient consensus protocols. In *Proceedings of the 2nd Annual ACM Symposium on Principles of Distributed Computing* (Montreal, Quebec, Canada, Aug. 17-19). ACM, New York, 1983, pp. 12-26.
- [5] DEMILLO, R. A., LYNCH, N. At, AND MERRITT, M. J. Cryptographic protocols. In *Proceedings of the 14th Annual ACM Symposium on Theory of Computing* (San Francisco, Calif., May 5-7). ACM, New York, 1982, pp. 383-400.
- [6] DOLEV, D., AND STRONG, H. R. Distributed commit with bounded waiting. In *Proceedings of the 2nd Annual IEEE Symposium on Reliability in Distributed Software and Database Systems*. IEEE, New York, 1982, pp. 53-60.
- [7] DOLEV, D., AND STRONG, H. R. Polynomial algorithms for multiple processor agreement. In *Proceedings of the 14th Annual ACM Symposium on Theory of Computing* (San Francisco, Calif., May 5-7). ACM, New York, 1982, pp. 401-407.
- [8] DOLEV, D., FISCHER, M., FOWLER, R., LYNCH, N., AND STRONG, H. R. An efficient algorithm for Byzantine agreement without authentication. *Inf. Control* 52, 3 (1983), 257-274.
- [9] DOLEV, D., LYNCH, N., PINTER, S., STARK, E., AND WEIHL, W. Reaching approximate agreement in the presence of faults. In *Proceedings of the 3rd Annual IEEE Symposium on Reliability in Distributed Software and Database Systems*. IEEE, New York, 1983, pp. 145-154.
- [10] DWORK, C., LYNCH, N., AND STOCKMEYER, L. Consensus in the presence of partial synchrony. In *Proceedings of the 3rd Annual ACM Symposium on Principles of Distributed Computing* (Vancouver, B.C., Canada, Aug. 27-29). ACM, New York, 1984, pp. 103-118.
- [11] FISCHER, M., AND LYNCH, N. A lower bound for the time to assure interactive consistency. *Inf. Proc. Lett.* 14, 4 (1982), 183-186.
- [12] FISCHER, M., LYNCH, N., AND PATERSON, M. Impossibility of distributed consensus with one faulty process. In *Proceedings of the 2nd Annual ACM SIGACT-SIGMOD Symposium on Principles of Database Systems* (Atlanta, Ga., Mar. 21-23). ACM, New York, 1983, pp. 1-7.
- [13] GARCIA-MOLINA, H. Elections in a distributed computing system. *IEEE Trans. Comput.* C-31, 1 (1982), 48-59.
- [14] LAMPORT, L., SHOSTAK, R., AND PEASE, M. The Byzantine Generals problem. *ACM Trans. Prog. Lang. syst.* 4, 3 (July 1982), 382-401.
- [15] LAMPSON, B. Replicated Commit. CSL Notebook Entry, Xerox Palo Alto Research Center, Palo Alto, Calif., 1981.
- [16] LAMPSON, B., AND STURGIS, H. Crash recovery in a distributed data storage system. Manuscript, Xerox Palo Alto Research Center, Palo Alto, Calif., 1979.
- [17] LINDSAY, B. G., SELINGER, P. G., GALTIERI, C., GRAY, J. N., LORIE, R. A., PRICE, T. G., PUTZOLU, F., TRAIGER, I. L., AND WADE, B. W. Notes on distributed databases. IBM Res. Rep. RJ2571, IBM Research Division, San Jose, Calif., 1979.
- [18] LYNCH, N., FISCHER, M., AND FOWLER, R. A simple and efficient Byzantine Generals algorithm. In *Proceedings of the 2nd Annual IEEE Symposium on Reliability in Distributed Software and Database Systems*. IEEE, New York, 1982, pp. 46-52.
- [19] PEASE, M., SHOSTAK, R., AND LAMPORT, L. Reaching agreement in the presence of faults. *J. ACM* 27, 2 (Apr. 1980), 228-234.
- [20] RABIN, M. Randomized Byzantine Generals. In *Proceedings of the 24th Annual IEEE Symposium on Foundations of Computer Science*. IEEE, New York, 1983, pp. 403-409.
- [21] REED, D. Naming and synchronization in a decentralized computer system. Ph.D. dissertation, Technical Report MIT/LCS/TR-205, Massachusetts Institute of Technology, Cambridge, Mass., 1978.
- [22] ROSENKRANTZ, D. J., STEARNS, R. E., AND LEWIS, P. M., II. System level concurrency control for distributed database systems. *ACM Trans. Database Syst.* 3, 2 (June 1978), 178-198.
- [23] SKEEN, D. A decentralized termination protocol. In *Proceedings of the 2nd Annual IEEE Symposium on Reliability in Distributed Software and Database Systems*. IEEE, New York, 1982, pp. 27-32.
- [24] SKEEN, D., AND STONEBRAKER, M. A formal model of crash recovery in a distributed system. *IEEE Trans. Softw. Engineering* SE-9, 3 (May 1983), 219-228.
- [25] TOUEG, S. Randomized Byzantine Agreements. In *Proceedings of the 3rd Annual ACM Symposium on Principles of Distributed Computing* (Vancouver, B.C., Canada, Aug. 27-29). ACM, New York, 1984, PP. 163-178.

Received September 1983; revised October 1984; accepted October 1984

参考文献

- [1] ATTIYA, C., DOLEV, D., GIL, J. 异步拜占庭共识。载于《第三届 ACM 分布式计算原理研讨会论文集》(加拿大不列颠哥伦比亚省温哥华, 8月27–29日)。ACM, 纽约, 1984年, 第119–133页。
- [2] BEN-OR, M. 自由选择的另一优势: 完全异步共识协议。载于《第二届 ACM 分布式计算原理研讨会论文集》(加拿大魁北克省蒙特利尔, 8月17–19日)。ACM, 纽约, 1983年, 第27–30页。
- [3] BRACHA, G. 一种异步的 $[(1)/3]$ -容错共识协议。载于《第三届 ACM 分布式计算原理研讨会论文集》(加拿大不列颠哥伦比亚省温哥华, 8月27–29日)。ACM, 纽约, 1984年, 第154–162页。
- [4] BRACHA, G., TOUEG, S. 具容错能力的共识协议。载于《第二届 ACM 分布式计算原理研讨会论文集》(加拿大魁北克省蒙特利尔, 8月17–19日)。ACM, 纽约, 1983年, 第12–26页。
- [5] DEMILLO, R. A., LYNCH, N. A., MERRITT, M. J. 密码学协议。载于《第十四届 ACM 计算理论年会论文集》(加利福尼亚州旧金山, 5月5–7日)。ACM, 纽约, 1982年, 第383–400页。
- [6] DOLEV, D., STRONG, H. R. 有界等待的分布式提交。载于《第二届 IEEE 分布式软件与数据库系统可靠性研讨会论文集》。IEEE, 纽约, 1982年, 第53–60页。
- [7] DOLEV, D., STRONG, H. R. 多处理器一致性的多项式算法。载于《第十四届 ACM 计算理论年会论文集》(加利福尼亚州旧金山, 5月5–7日)。ACM, 纽约, 1982年, 第401–407页。
- [8] DOLEV, D., FISCHER, M., FOWLER, R., LYNCH, N., STRONG, H. R. 一种无需认证的高效拜占庭协议算法。《信息与控制》第52卷第3期(1983年), 257–274页。
- [9] DOLEV, D., LYNCH, N., PINTER, S., STARK, E., WEIHL, W. 故障存在条件下的近似一致性达成。载于《第三届 IEEE 分布式软件与数据库系统可靠性研讨会论文集》。IEEE, 纽约, 1983年, 第145–154页。
- [10] DWORK, C., LYNCH, N., STOCKMEYER, L. 在部分同步条件下达成共识。载于《第三届 ACM 分布式计算原理研讨会论文集》(加拿大不列颠哥伦比亚省温哥华, 8月27–29日)。ACM, 纽约, 1984年, 第103–118页。
- [11] FISCHER, M., LYNCH, N. 保证交互一致所需时间的下界。《信息处理快报》第14卷第4期(1982年), 183–186页。
- [12] FISCHER, M., LYNCH, N., PATERSON, M. 带有一个错误进程的分布式共识不可能性。载于《第二届 ACM SIGACT-SIGMOD 数据库系统原理研讨会论文集》(佐治亚州亚特兰大, 3月21–23日)。ACM, 纽约, 1983年, 第1–7页。
- [13] GARCIA-MOLINA, H. 分布式计算系统中的选举。《IEEE 计算机汇刊》C-31卷第1期(1982年), 48–59页。
- [14] LAMPORT, L., SHOSTAK, R., PEASE, M. 拜占庭将军问题。《ACM 程序设计语言与系统汇刊》第4卷第3期(1982年7月), 382–401页。
- [15] LAMPSON, B. 复制提交。CSL 笔记条目, 施乐帕洛阿尔托研究中心, 加利福尼亚州帕洛阿尔托, 1981年。
- [16] LAMPSON, B., STURGIS, H. 分布式数据存储系统中的崩溃恢复。手稿, 施乐帕洛阿尔托研究中心, 加利福尼亚州帕洛阿尔托, 1979年。
- [17] LINDSAY, B. G., SELINGER, P. G., GALTIERI, C., GRAY, J. N., LORIE, R. A., PRICE, T. G., PUTZOLU, F., TRAIGER, I. L., AND WADE, B. W. 分布式数据库笔记。IBM研究报告RJ2571, IBM研究部, 加利福尼亚州圣何塞, 1979年。
- [18] LYNCH, N., FISCHER, M., AND FOWLER, R. 一种简单高效的拜占庭将军算法。载于《第2届IEEE分布式软件与数据库系统可靠性年会论文集》。IEEE, 纽约, 1982年, 第46–52页。
- [19] PEASE, M., SHOSTAK, R., AND LAMPORT, L. 故障存在下的共识达成。《ACM期刊》27, 2 (1980年4月), 第228–234页。
- [20] RABIN, M. 随机化拜占庭将军问题。载于《第24届IEEE计算机科学基础年会论文集》。IEEE, 纽约, 1983年, 第403–409页。
- [21] REED, D. 去中心化计算机系统命名与同步。博士论文, 技术报告MIT/LCS/TR-205, 麻省理工学院, 马萨诸塞州剑桥, 1978年。
- [22] ROSENKRANTZ, D. J., STEARNS, R. E., AND LEWIS, P. M., II. 分布式数据库系统的系统级并发控制。《ACM数据库系统汇刊》3, 2 (1978年6月), 第178–198页。
- [23] SKEEN, D. 一种去中心化的终止协议。载于《第2届IEEE分布式软件与数据库系统可靠性年会论文集》。IEEE, 纽约, 1982年, 第27–32页。
- [24] SKEEN, D., AND STONEBRAKER, M. 分布式系统中崩溃恢复的正式模型。《IEEE软件工程汇刊》SE-9, 3 (1983年5月), 第219–228页。
- [25] TOUEG, S. 随机化拜占庭协议。载于《第3届ACM分布式计算原理年会论文集》(加拿大不列颠哥伦比亚省温哥华, 8月27–29日)。ACM, 纽约, 1984年, 第163–178页。

1983年9月收稿; 1984年10月修订; 1984年10月接受